

UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE CIENCIAS, 2027-I
MODELADO Y PROGRAMACIÓN

PROYECTO 1:

Chat: Servidor - Cliente

MORALES GALVÁN MARÍA PAOLA

323073406

28/09/2026

PROFESOR:

Canek Peláez Valdés

AYUDANTE DE TEORÍA:

Cynthia Lizbeth Sánchez Urbano

AYUDANTE DE LABORATORIO:

Miriam Torres Bucio

Resumen

En este proyecto se diseñó e implementó un sistema de comunicación distribuido en tiempo real bajo la arquitectura cliente-servidor[cite: 1]. La solución se compone de un servidor desarrollado en Golang, encargado del control de concurrencia mediante *goroutines* y la gestión de conexiones de red, y un cliente desarrollado en C# orientado a la interacción desde la línea de comandos[cite: 1]. Ambos componentes se comunican mediante sockets sobre el protocolo TCP, utilizando tramas serializadas en formato JSON para la transferencia estructurada de mensajes, respuestas y notificaciones[cite: 1]. El sistema permite el intercambio de mensajes privados, difusiones grupales y la administración de salas independientes[cite: 1]. Cabe destacar que esta implementación se desarrolló con fines estrictamente académicos para la asignatura de Modelado y Programación, por lo que omite protocolos de seguridad o cifrado en la transmisión, y los datos presentados en el reporte son de carácter puramente ilustrativo[cite: 1].

Introducción

El proyecto aborda el diseño e implementación de un sistema de comunicación en tiempo real distribuido bajo el modelo cliente-servidor. El propósito fundamental es desarrollar una plataforma funcional de chat en consola que gestione conexiones concurrentes, sincronización de estados y un protocolo de mensajería.

Objetivos del Proyecto

Objetivo General:

- Diseñar e implementar un sistema distribuido de mensajería cliente-servidor tolerante a fallos, capaz de gestionar múltiples conexiones concurrentes y facilitar el intercambio de información estructurada en tiempo real.

Objetivos Específicos:

- Desarrollar un servidor de red capaz de aceptar y administrar múltiples clientes de forma simultánea, asociando a cada uno una unidad de ejecución independiente sin interferir con los demás usuarios.
- Implementar un cliente interactivo que permita al usuario enviar mensajes, unirse a salas y recibir notificaciones del servidor asíncronamente en tiempo real.
- Implementar un protocolo en formato JSON para normar las solicitudes, respuestas y notificaciones entre ambos componentes.

Presentación de Componentes

El sistema se compone de dos aplicaciones independientes pero estrechamente acopladas por su protocolo de red:

- **Servidor (Golang):** Encargado de la gestión de conexiones TCP, el control de concurrencia y el almacenamiento de datos en memoria (usuarios activos, invitaciones y salas de chat).
- **Cliente (C#):** Captura la entrada del usuario, construye las solicitudes correspondientes y despliega las notificaciones entrantes sin bloquear la interacción.
- **Entorno de Construcción (Meson):** Sistema de gestión encargado de compilar la aplicación cliente de forma automatizada.

Al tratarse de un desarrollo enfocado en el modelado de sistemas concurrentes el proyecto no incluye mecanismos de autenticación o seguridad en el canal de transmisión.

Analisis y Diseño (Modelo)

En esta sección desglosaremos como se hizo el analisis y diseño del proyecto
Utilizaremos el siguiente protocolo para el proyecto:

Solicitudes del cliente

- **IDENTIFY:** Para identificar al usuario con el servidor, con la propiedad *username*
- **STATUS:** Cambia el estado de un usuario por (ACTIVE, AWAY o BUSY), con la propiedad *status*
- **USERS:** Solicita la lista de usuarios en el chat
- **TEXT:** Manda un mensaje en privado a un usuario, con las propiedades *username*, *text*
- **PUBLIC_TEXT:** Manda un mensaje a todos los usuarios en el chat, con la propiedad *text*
- **NEW_ROOM:** Crea un cuarto privado, con la propiedad *roomname*
- **INVITE:** Invita usuarios al cuarto, con las propiedades *roomname*, *usernames*
- **JOIN_ROOM:** Une al usuario al cuarto siempre y cuando haya sido invitado previamente, con la propiedad *roomname*
- **ROOM_USERS:** Solicita la lista de usuarios en un cuarto, con la propiedad *roomname*
- **ROOM_TEXT:** Manda un mensaje dentro de un cuarto, con las propiedades *roomname*, *text*
- **LEAVE_ROOM:** Solicita dejar un cuarto, con la propiedad *roomname*
- **DISCONNECT:** Desconecta al usuario del chat

Notificaciones del servidor

- **RESPONSE:** Respuesta directa a una solicitud del cliente, con las propiedades *operation*, *result*, *extra*
- **NEW_USER:** Notifica que se unio un nuevo usuario al chat, con la propiedad *username*
- **NEW_STATUS:** Notifica que algun usuario actualizó su estado a (ACTIVE, AWAY o BUSY), con las propiedades *username*, *status*
- **USER_LIST:** Regresa la lista de usuarios del chat, con la propiedad *users*
- **TEXT_FROM:** Notifica que algun usuaio mando un mensaje en privado, con las propiedades *username*, *text*
- **PUBLIC_TEXT_FROM:** Notifica que algun usuario mando un mensaje al chat grupal, con las propiedades *username*, *text*
- **JOINED_ROOM:** Notifica que un usuario se unio a un cuarto, con las propiedades *roomname*, *username*
- **ROOM_USER_LIST:** Regresa la lista de usuarios en un cuarto, con las propiedades *roomname*, *users*
- **ROOM_TEXT_FROM:** Notifica que algun usuario mando un mensaje a un cuarto, con las propiedades *roomname*, *username*, *text*
- **LEFT_ROOM:** Notifica que un usuario abandono un cuarto, con las propiedades *roomname*, *username*
- **DISCONNECTED:** Notifica que un usuario se desconecto del chat, con la propiedad *username*

Respuestas del servidor

- **SUCCESS:** Indica que se ejecuto la acción con exito
- **USER_ALREADY_EXISTS:** Indica que el nombre de usuario ya existe en el servidor
- **NO_SUCH_USER:** Indica que el usuario no existe en el chat
- **ROOM_ALREADY_EXISTS:** Indica que ya existe un cuarto con el nombre
- **NO_SUCH_ROOM:** Indica que el cuarto no existe
- **NOT_INVITED:** Indica que no has sido invitado al cuarto
- **NOT_JOINED:** Indica que el usuario no se ha unido al cuarto
- **NOT_IDENTIFIED:** Indica que el usuario no se ha identificado

Propiedades

- **username:** Nombre de usuario del cliente
- **status:** Estado del cliente
- **text:** Mensaje que el cliente quiere enviar
- **roomname:** Nombre del cuarto
- **usernames:** Nombre de los usuarios
- **users:** Lista de usuarios
- **operation:** Solicitud que se intento realizar
- **result:** Resultado de la solicitud
- **extra:** Información extra, usualmente el nombre de usuario o sala

Análisis

El proyecto consiste en el desarrollo de un sistema de chat en tiempo real. Este problema se puede descomponer en dos partes fundamentales: un servidor capaz de gestionar múltiples clientes de forma simultánea y un cliente capaz de conectarse de manera independiente a dicho servidor. Ambos componentes presentan una dependencia mutua estricta para garantizar el funcionamiento del sistema. Para lograr esto, es imprescindible que cada cliente conectado disponga de su propio hilo o unidad de ejecución concurrente en el servidor, permitiendo procesar distintas acciones simultáneamente.

Por un lado, el servidor debe cumplir con las siguientes funciones clave:

- Recibir y gestionar conexiones entrantes.
- Mantener la concurrencia activa entre conexiones sin interferir con la operación de otros clientes.
- Almacenar y administrar la información de los clientes y las salas en memoria.
- Recibir e interpretar mensajes entrantes en formato JSON.
- Construir y enviar mensajes de respuesta y notificación en formato JSON.
- Ejecutar las acciones solicitadas por cada cliente de manera aislada.
- Manejar los posibles errores y fallos de comunicación de forma limpia sin interrumpir el servicio.

A partir de estas responsabilidades, se establece que el servidor debe aceptar todas las conexiones válidas. En caso de recibir una conexión inválida o corrupta, debe desconectarla inmediatamente tras detectar el error, sin que esto afecte el comportamiento ni la estabilidad del sistema. El servidor debe mantener la conexión activa con el cliente hasta que este solicite explícitamente su desconexión o hasta que ocurra un error irrecuperable en el canal de red. Mantener la conexión implica procesar correctamente cada solicitud del cliente, emitiendo las respuestas e iterando las notificaciones correspondientes a los demás usuarios.

Es un requisito fundamental que el servidor sea tolerante a fallos e "inmortal", lo que significa que, independientemente de las solicitudes malformadas, la conducta de los clientes o los errores puntuales en las conexiones, el servidor debe continuar operando para el resto de los usuarios conectados. Por último, debe ser capaz de administrar múltiples clientes concurrentes sin incurrir en bloqueos mutuos (deadlocks) ni pérdida de información.

Por otro lado, el cliente debe cumplir con las siguientes responsabilidades:

- Establecer la conexión con el servidor a partir de la dirección IP y el puerto proporcionados.
- Escuchar continuamente los mensajes enviados por el servidor y las peticiones ingresadas por el usuario.
- Interpretar las acciones solicitadas por el usuario desde la interfaz de comandos.
- Construir y enviar mensajes en formato JSON hacia el servidor.

Dado que el usuario no debe escribir mensajes JSON manualmente, el cliente debe interpretar las órdenes ingresadas por el usuario, transformarlas y construir la estructura JSON correspondiente antes de transmitirla. Asimismo, debe ser capaz de enviar solicitudes y recibir notificaciones o respuestas de forma simultánea, gestionando esta asincronía de manera robusta para evitar fallos en la aplicación. Finalmente, todas las notificaciones recibidas desde el servidor deben ser formateadas y desplegadas al usuario de una manera intuitiva y legible.

Ambos componentes (cliente y servidor) deben acordar un esquema coherente de codificación y decodificación de mensajes JSON para evitar la lectura de parámetros inválidos; por ejemplo, si el servidor envía un valor de tipo cadena (**string**), el cliente debe decodificarlo explícitamente bajo el mismo tipo de dato.

Diseño

La elección de **Golang** para la implementación del servidor se fundamenta en su modelo de concurrencia nativo basado en *goroutines*, el cual permite gestionar múltiples conexiones TCP simultáneas con un consumo mínimo de recursos, sumado a la simplicidad del paquete `net` para la creación de *listeners* y *sockets*.

Por su parte, el lenguaje **C#** para el desarrollo del cliente se seleccionó debido a sus librerías de red nativas, las cuales permiten crear y administrar *sockets* de forma directa

e intuitiva, ofreciendo además herramientas integradas para la serialización y deserialización de datos entre JSON u objetos de C#. Adicionalmente, se integró el sistema de construcción **Meson** al cliente debido a su facilidad en la administración de dependencias y a que permite definir reglas de compilación claras en su archivo de configuración (`meson.build`), generando el ejecutable del proyecto sin necesidad de ejecutar comandos complejos.

A continuación, se detalla la comunicación e interacción entre el servidor y el cliente según lo establecido en la etapa de análisis.

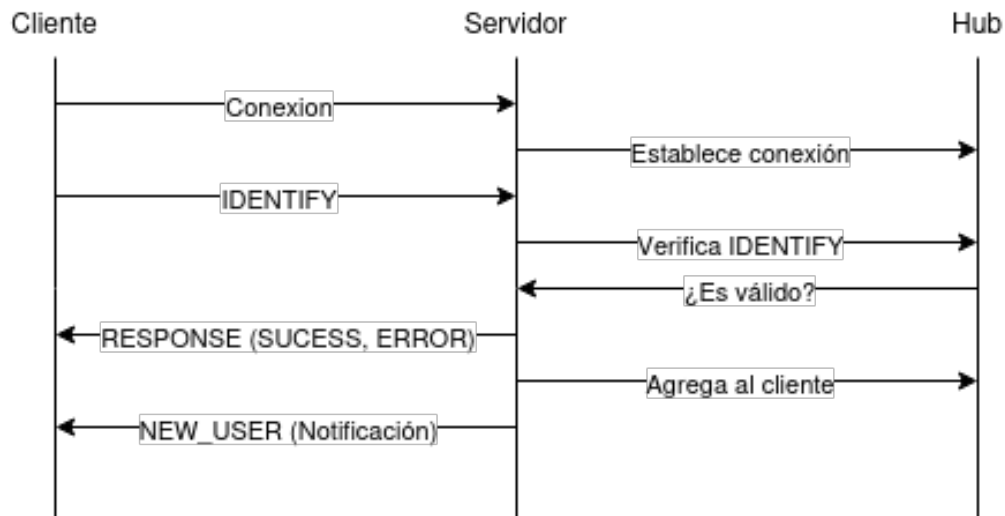


Figura 1: Diagrama de conexión e identificación del cliente.

Como se observa en el diagrama, el cliente inicia el proceso intentando conectarse al servidor. Una vez establecida la conexión, el cliente debe identificarse enviando la solicitud *IDENTIFY*. Si la identificación es válida, el servidor registra al usuario y emite la notificación *NEW_USER* al resto de los clientes conectados. Cabe destacar que todos los mensajes recibidos por el servidor siguen un mismo ciclo de procesamiento: recepción, validación, interpretación y ejecución. Si alguno de estos pasos falla, el cliente en cuestión es desconectado para prevenir fallos globales y preservar la ejecución del servidor.

El siguiente diagrama ejemplifica el funcionamiento y las estructuras internas del servidor:

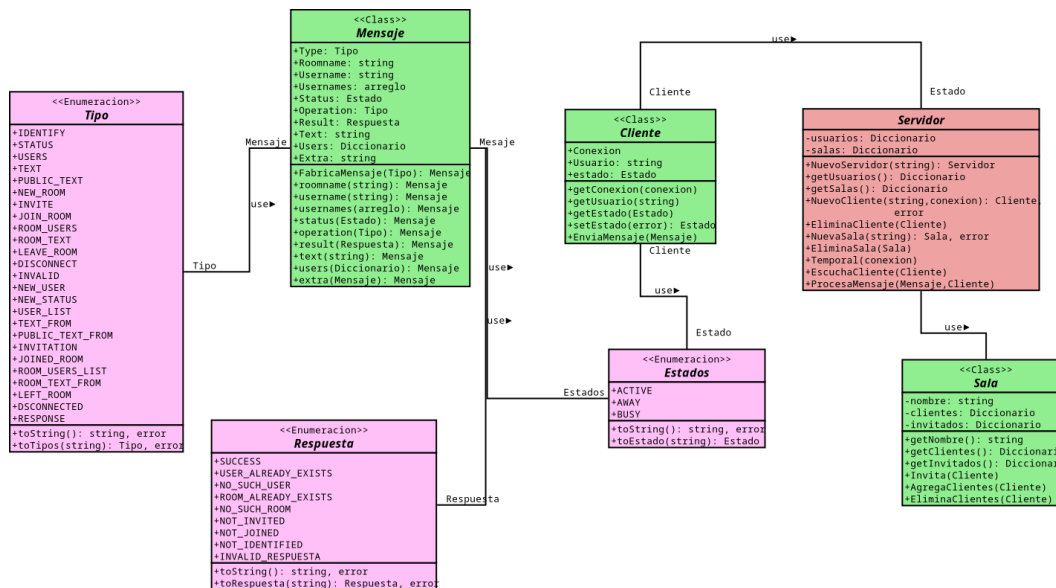


Figura 2: UML del servidor.

Como se observa, el servidor mantiene dos propiedades principales representadas como mapas/diccionarios:

- **Usuarios:** Almacena el nombre de usuario como clave y una referencia a la estructura **Cliente** como valor.
- **Salas:** Almacena el nombre de la sala como clave y una referencia a la estructura **Sala** correspondiente.

Estas estructuras permiten preservar el registro global de todos los usuarios y salas existentes en el sistema.

Por su parte, la abstracción del **Cliente** en el servidor se compone de tres propiedades:

- **Usuario:** Nombre identificador elegido por el cliente.
- **Conexión:** Referencia al socket o identificador del cliente dentro de la red.
- **Estado:** Valor perteneciente a la enumeración de estados permitidos, cuyo valor predeterminado al conectarse es **ACTIVE**.

Las **Salas** contienen tres propiedades fundamentales:

- **Nombre:** Nombre identificador de la sala.
- **Clientes:** Diccionario que asocia el nombre del usuario (*string*) con su respectiva referencia de **Cliente**.
- **Invitados:** Diccionario con la misma estructura que el de clientes, utilizado para llevar el control de los usuarios que han sido invitados a la sala.

La estructura genérica de los **Mensajes** dispone de 10 propiedades que corresponden exactamente a los campos definidos en el protocolo JSON, garantizando una estructura homogénea para la transferencia de datos.

Adicionalmente, se implementaron tres enumeraciones principales (TIPO, RESPUESTA y ESTADO). Esta decisión de diseño permite que, al recibir un mensaje en formato JSON, el sistema pueda validar e identificar de forma limpia si la operación o el contenido es válido.

A continuación, se ejemplifica el funcionamiento interno del cliente:

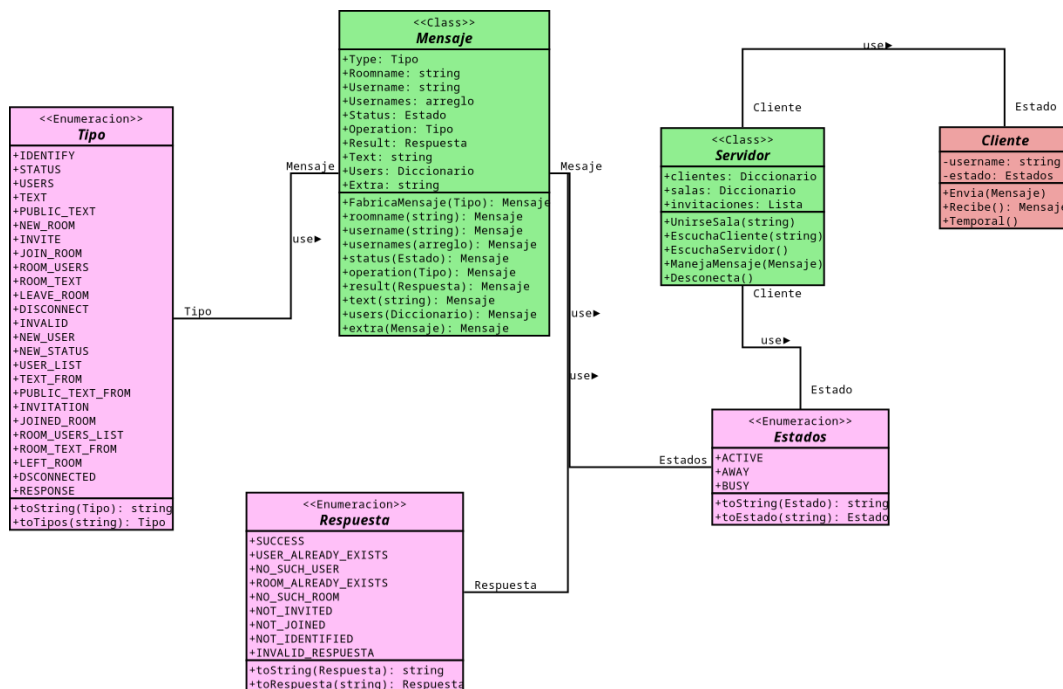


Figura 3: UML del cliente.

Como se puede observar, el cliente mantiene tres propiedades principales:

- **Usuario:** Nombre del usuario identificado.
- **Estado:** Estado actual del usuario.
- **Conexión:** Instancia de la conexión por socket con el servidor.

Para la administración del estado local, la abstracción del entorno del cliente conserva:

- Una referencia al cliente.
- Un diccionario de clientes (clave: *string*, valor: *Estado*).
- Un diccionario de salas (clave: *string*, valor: *Lista de cadenas*).
- Una lista de invitaciones recibidas.

Lo anterior evita la necesidad de solicitar constantemente la lista completa de usuarios al servidor, permitiendo además acceder localmente a la información de las salas a las que se pertenece o a las que se ha sido invitado.

Al igual que en el servidor, los mensajes en el cliente contienen las propiedades definidas por el protocolo para asegurar el formato de las tramas serializadas. Se incluyen las enumeraciones Tipo, Respuesta y Estado para validar las tramas entrantes y garantizar el correcto armado de los JSON salientes.

Implementación

En esta sección se describen las tecnologías específicas, bibliotecas, mecanismos de sincronización y métodos clave que hacen funcional la arquitectura del sistema de chat.

Tecnologías Utilizadas

- **Servidor (Golang):** Implementado en Go utilizando su librería nativa `net` para el manejo de conexiones TCP sobre sockets, `encoding/json` para la serialización y deserialización de mensajes, y el paquete `sync` para el control de concurrencia.
- **Cliente (C#):** Desarrollado en C# (.NET) utilizando `System.Net.Sockets` para la gestión del socket de red, `System.Text.Json` para el parseo de estructuras de mensajes y hilos/tareas asíncronas (`Task.Run`) para no bloquear la consola durante la lectura.
- **Administración del Proyecto (Meson):** Se incorporó el sistema de construcción Meson para definir el flujo de compilación del cliente de C# a través del archivo `meson.build`.

Detalles y Métodos Clave del Servidor (Golang)

El servidor se diseñó para procesar clientes de forma concurrente protegiendo las estructuras globales en memoria (`Usuarios` y `Salas`) mediante cerrojos de exclusión mutua (`sync.RWMutex`).

- **Método Temporal:** Controla la conexión inicial de un cliente antes de que este se identifique. Escucha el socket entrante y valida que el primer mensaje sea un comando `IDENTIFY`. Si el nombre de usuario ya está registrado en la estructura global o el mensaje es inválido, emite la respuesta de error y cierra el socket inmediatamente, evitando el desperdicio de recursos.
- **Métodos de Gestión de Memoria (AgregarCliente, EliminarCliente, AgregarSala, EliminarSala):** Métodos encargados de la modificación de las estructuras de mapas en el servidor. Dado que en Go no es seguro modificar un mapa mientras se iteran sus elementos desde múltiples *goroutines*, estos métodos encierran las operaciones entre bloques `Lock()` y `Unlock()` del cerrojo `RWMutex`, asegurando modificaciones atómicas.

- **Gestión de Conexiones Activas:** Por cada conexión aceptada en el socket principal, el servidor lanza una nueva *goroutine* que atiende de forma asíncrona la lectura continua de tramas JSON enviadas por dicho cliente.

Detalles y Métodos Clave del Cliente (C#)

El cliente gestiona la asincronía entre la lectura del teclado por parte del usuario y la recepción de notificaciones de red enviadas por el servidor.

- **Método Temporal:** Maneja la rutina de conexión del cliente. Solicita los datos de red (IP y puerto), establece la conexión socket mediante `TcpClient` y envía el paquete de identificación `IDENTIFY`. Al recibir una respuesta exitosa (`SUCCESS`), inicializa las estructuras locales y solicita automáticamente la lista actualizada de usuarios (`USERS`).
- **Método EscuchaCliente:** Se ejecuta en el hilo principal de la aplicación. Mantiene un bucle continuo capturando la entrada ingresada por el usuario en la terminal, traduce la orden a su representación JSON equivalente usando el protocolo acordado y la transmite por el socket.
- **Método EscuchaServidor:** Corre en un hilo secundario asíncrono (`Task.Run`). Permanece a la espera de bytes en el socket, deserializa las tramas JSON entrantes y despliega las notificaciones (`TEXT_FROM`, `NEW_USER`, `ROOM_TEXT_FROM`, etc.) formateadas en la consola sin interrumpir la escritura activa del usuario.

Conclusiones

El desarrollo de este proyecto me permitió integrar de forma práctica los conceptos de la programación concurrente, el diseño de sistemas distribuidos bajo el modelo cliente-servidor y la manipulación de sockets de red. La utilización de *goroutines* y la simplicidad del paquete `net` considero que fueron un acierto para la construcción del servidores, ya que me facilitaron la atención simultánea de múltiples clientes mediante hilos. Asimismo, logre notar la importancia de aplicar un control estricto de exclusión mutua mediante cerrojos de lectura y escritura para proteger las estructuras compartidas en memoria.

Por otro lado, en la implementación del cliente, la ejecución asíncrona mediante tareas en segundo plano en C# permitió procesar las notificaciones del servidor en tiempo real sin congelar ni bloquear la captura de comandos ingresados desde la terminal. De igual forma, la integración del sistema de construcción Meson simplificó de manera notable la administración de dependencias y el flujo de compilación del cliente.

En cuanto a las dificultades encontradas durante el desarrollo, uno de los mayores retos radicó en la gestión precisa de los cerrojos de exclusión mutua (`sync.RWMutex`) para prevenir la aparición de interbloqueos (*deadlocks*) al acceder y modificar concurrentemente las estructuras. Asimismo, la correcta implementación del protocolo representó un reto significativo, ya que requirió garantizar que los mensajes en formato JSON fueran serializadas y deserializadas de manera simétrica y sin ambigüedades.

Bibliografía

1. The Go Programming Language Documentation. *Package net: Sockets TCP, Listeners y manejo de conexiones de red*. Disponible en: <https://pkg.go.dev/net>
2. The Go Programming Language Documentation. *Package sync: Cerrojos Mutex (RWMutex) y control de concurrencia*. Disponible en: <https://pkg.go.dev/sync>
3. The Go Programming Language Documentation. *Package encoding/json: Serialización y deserialización de datos*. Disponible en: <https://pkg.go.dev/encoding/json>
4. Microsoft .NET Documentation. *System.Net.Sockets Namespace: Manejo de sockets y conexiones TCP en C#*. Disponible en: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets>
5. Microsoft .NET Documentation. *System.Text.Json Namespace: Serialización y mapeo de objetos JSON en C#*. Disponible en: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json-overview>
6. Microsoft .NET Documentation. *Asynchronous programming with async and await / Task.Run in C#*. Disponible en: <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/>
7. The Meson Build System Documentation. *Meson Build System Reference Manual y sintaxis de meson.build*. Disponible en: <https://mesonbuild.com/>
8. IETF RFC 793. *Transmission Control Protocol (TCP) Specification: Asignación de puertos efímeros, estados de conexión y ciclo de vida de sockets*. Disponible en: <https://datatracker.ietf.org/doc/html/rfc793>
9. IETF RFC 8259. *The JavaScript Object Notation (JSON) Data Interchange Format*. Disponible en: <https://datatracker.ietf.org/doc/html/rfc8259>
10. Tanenbaum, A. S., & Wetherall, D. J. *Redes de Computadoras: Arquitectura Cliente-Servidor y Capa de Transporte*. 5ta Edición, Pearson Education.